

Linux Backup System

Full and incremental backups, secure transfer over SSH
and safe restore: from idea to daily use

Bash

tar

SSH / SCP

SHA-256

cron

Contents

1 · Architecture: how the project is built	3
2 · Prerequisites.....	6
3 · Usage, step by step with examples	7
4 · Errors and troubleshooting.....	11

What problem does it solve?

If a disk fails, files are deleted or malware reaches them, everything is lost. A backup that stays on the same machine disappears at the same moment. So a healthy copy must live somewhere else.

The solution at a glance

Job 1

Build

Create a compressed archive of the data folder: a full one, or only the changes

Job 2

Send

Deliver it securely to a server and re-check its integrity there

Job 3

Restore

When needed, bring the data back safely and predictably

Goal of this guide

After reading it you should be able to explain what each part does, why it exists and how to use it.



Architecture

How the project is built, and why

The path of one backup, from Debian to Kali

1 · BUILD

tar + sha256

Compress the data and create the file's fingerprint

2 · SEND

scp over SSH

Ed25519 key, no password: suitable for cron

3 · QUARANTINE

incoming/

The file has arrived but is not trusted yet

4 · STORE

store/

Only after the server itself verified the checksum

One command, `run.sh`, runs these stages in order and then removes very old backups. Each stage starts only if the previous one succeeded.

Stage	If it fails
<code>backup.sh</code>	Transfer does not start; exit code 10
<code>transfer.sh</code>	The good local backup stays and the next run retries (exit code 20 or 21)
<code>prune.sh</code>	No backup is lost; only the cleanup is skipped (exit code 30)

Why a single command?

If the backup is scheduled at 02:00 and the transfer at 02:30, a backup that takes 31 minutes would be sent half-written. `run.sh` removes that problem at the root.

Project components

```
linux-backup-project/
├─ bin/      run.sh  backup.sh  transfer.sh  restore.sh  prune.sh
├─ lib/      common.sh                # config, logging, lock, exit codes
├─ config/   backup.conf              # the only file you edit
├─ server/   backup-gate.sh           # installed on the server
├─ tools/    report.py                # report and health check
├─ tests/    selftest.sh              # automated test
└─ data/     backups/ state/ logs/    # runtime folders
```

What each script does

File	Its job in plain words
<code>run.sh</code>	The coordinator: backup, then transfer, then cleanup. This is the one that goes into cron
<code>backup.sh</code>	Creates a <code>full</code> , <code>incremental</code> or <code>auto</code> archive plus a <code>.sha256</code> file next to it
<code>transfer.sh</code>	Uploads archives that were not sent yet and waits for the server's confirmation
<code>restore.sh</code>	Lists backups and restores data into a new folder
<code>prune.sh</code>	Keeps the newest few "sets" and deletes older ones
<code>backup-gate.sh</code>	The server's guard: accepts only three kinds of request
<code>report.py</code>	(Python) lists backups and verifies checksums

Three key concepts

1. Full and incremental backups

A **full** backup copies everything. An **incremental** backup copies only files that are new or changed since the previous run, and also records deletions. A **set** is one full backup plus the incrementals that follow it.



To restore, the full backup and every incremental after it must be intact and applied in order. That is why sets are always deleted as a whole.

2. Checksum, the file's fingerprint

SHA-256 is a number computed from a file's content. If a single byte changes, the number changes. The project checks it four times:



3. A restricted key and a guard (the gate)

The SSH key has no passphrase so cron can use it. If it is stolen it must be almost useless. This is the principle of **least privilege**.

Allowed	Forbidden
<code>ping</code> to test the connection	opening a shell or running commands
upload files only into <code>incoming/</code>	reading, listing or downloading backups
<code>verify</code> to confirm a checksum	deleting or overwriting stored backups

Why does a new file go to incoming first?

If someone steals the key and uploads a malicious file named like an old backup, it stays in quarantine. A file already in `store/` is never overwritten, and a new file enters `store/` only with a valid checksum.

2

Prerequisites

What must be ready before you start

Item	Debian client · 192.168.206.129	Kali server · 192.168.206.131
Role	Scripts and data live here; backups are created here	The destination; received backups are stored here
Software	bash tar gzip sha256sum openssh-client cron python3	openssh-server running sha256sum scp
User	A normal user (youngev)	A dedicated user bkpuser
Folders	~/linux-backup-project	/srv/backups/myhost/incoming /srv/backups/myhost/store
Special files	config/backup.conf and key ~/.ssh/backup_ed25519	/usr/local/bin/backup-gate.sh and authorized_keys

Quick checklist

1) The two machines can see each other. On Debian, you should see 0% packet loss:

```
ping -c 2 192.168.206.131
```

2) SSH is running on Kali:

```
sudo systemctl status ssh --no-pager # should say active (running)
# if it is not installed
sudo apt install openssh-server && sudo systemctl enable --now ssh
```

3) Do not name the backup user backup. That name is already a system account on Debian and Kali. We created bkpuser instead.

3

Usage

One-time setup, then daily use

Step 1 · Prepare the server Kali

```
sudo adduser bkpuser
sudo mkdir -p /srv/backups/myhost/incoming /srv/backups/myhost/store
sudo chown -R bkpuser:bkpuser /srv/backups/myhost
```

Step 2 · Send the gate script to the server

On Debian:

```
cd ~/linux-backup-project
scp server/backup-gate.sh youngev@192.168.206.131:/tmp/
```

Then on Kali, owned by root so the backup user cannot edit it:

```
sudo install -o root -g root -m 755 /tmp/backup-gate.sh /usr/local/bin/backup-gate.sh
```

Step 3 · Create the key Debian

```
ssh-keygen -t ed25519 -f ~/.ssh/backup_ed25519 -C "backup@$(hostname)" -N ""
chmod 600 ~/.ssh/backup_ed25519
scp ~/.ssh/backup_ed25519.pub youngev@192.168.206.131:/tmp/
```

Send only the `.pub` file

The file without `.pub` is the private key and must never leave its machine.

Step 4 · Enable the key, with restrictions

Kali

```
sudo mkdir -p /home/bkpuser/.ssh
echo "restrict,command=\"/usr/local/bin/backup-gate.sh\",from=\"192.168.206.129\" $(cat /
tmp/backup_ed25519.pub)" \
| sudo tee /home/bkpuser/.ssh/authorized_keys
sudo chown -R bkpuser:bkpuser /home/bkpuser/.ssh
sudo chmod 700 /home/bkpuser/.ssh && sudo chmod 600 /home/bkpuser/.ssh/authorized_keys
sudo usermod -p '*' bkpuser          # disable the password; key-only from now on
```

Option	Meaning
<code>restrict</code>	Switches off every extra: terminal, port forwarding, X11
<code>command=...</code>	Whatever the client asks for, only this script runs
<code>from=...</code>	The key works only from the client's IP (optional)

Step 5 · Test the restrictions

Debian

```
ssh -i ~/.ssh/backup_ed25519 bkpuser@192.168.206.131 ping
# → pong
ssh -i ~/.ssh/backup_ed25519 bkpuser@192.168.206.131 ls /
# → backup-gate: command not allowed
```

If you get `pong` and `ls` is refused, the lock works.

Step 6 · Project configuration

Debian

```
cd ~/linux-backup-project
cp config/backup.conf.example config/backup.conf
sed -i 's|^REMOTE_USER=.*|REMOTE_USER="bkpuser"|; s|^REMOTE_HOST=.*|
REMOTE_HOST="192.168.206.131"|' config/backup.conf
```

The other values (port 22, key path, keeping 4 sets) are defaults and are correct.

First run and daily use Debian

```
echo "hello backup" > data/a.txt
bin/run.sh full # full backup + upload + cleanup
```

The next day, only the changes are captured. Real output of one run:

```
[backup ] START type=incremental file=daily_2026-10-09_12-46-47_incremental.tar.gz
[backup ] RESULT status=SUCCESS type=incremental size=206B duration=0s
[transfer] Uploading daily_2026-10-09_12-46-47_incremental.tar.gz (206B)
[transfer] RESULT status=SUCCESS stage=remote-verify (verified by server)
[prune ] RESULT status=SUCCESS prune kept_sets=2 deleted_archives=0
[run ] Pipeline finished OK
```

Command	What it does
<code>bin/run.sh auto</code>	One full backup per week, incrementals on the other days (for cron)
<code>bin/run.sh full</code>	Force a full backup
<code>bin/restore.sh --list</code>	Show all backups grouped into sets
<code>python3 tools/report.py</code>	Check the health of the checksums
<code>bin/prune.sh --dry-run</code>	Only show what would be deleted
<code>tail logs/backup.log</code>	View the log

Automatic runs with cron

```
crontab -e
# every night at 02:00
0 2 * * * /home/youngev/linux-backup-project/bin/run.sh auto >> /home/youngev/linux-backup-project/logs/cron.log 2>&1
```

A single line. `run.sh` itself keeps the stages in order.

Restore Debian

```
bin/restore.sh --list           # what is available?
bin/restore.sh --to /tmp/restored # latest state
bin/restore.sh --to /tmp/restored --backup 2026-10-09_12-43 # a point in time
bin/restore.sh --to /tmp/restored --dry-run           # show the plan only
diff -r data /tmp/restored/data && echo "restore OK"
```

If `diff` prints nothing and you see `restore OK`, the restored data is identical to the current data.

What happens behind the scenes?

#	Step
1	The target archive is found; if it is an incremental, the full backup before it and every incremental in between are selected too
2	Before anything else: the chain is complete, and every archive passes SHA-256 and gzip checks
3	The destination must be empty; no existing file is ever overwritten
4	Extraction into a temporary folder, in order: full, then incrementals
5	Only if everything succeeded is the temporary folder moved to the destination; otherwise it is deleted and the destination stays untouched

Restoring from the server

The backup key can only "write" on purpose, so it cannot download anything. To restore, log in to Kali with your own account, take the archives from `store/`, then point `restore.sh --dir` at that folder.

4

Errors and troubleshooting

The exit code tells you exactly what broke

Code	Meaning	Script
0	Success	all
3	Incomplete or wrong configuration	all
4	Another job is running (lock)	all
10	Backup creation failed; no transfer is attempted	backup / run
20	Upload failed: network, SSH or scp; retried next run	transfer / run
21	Checksum mismatch, local or on the server	transfer / run
30	Cleanup of old backups failed	prune / run
40	Restore refused before touching anything	restore
41	Extraction failed; temporary files were removed	restore

To find every failure in the log: `grep 'status=FAILED' logs/backup.log`

Common problems

Symptom	What to check
Permission denied (publickey)	The <code>SSH_KEY</code> path; the <code>authorized_keys</code> entry must be a single line; permissions 700 and 600
Host key verification failed	Run <code>ssh ... ping</code> once by hand and accept the fingerprint
Exit code 20 but manual SSH works	Is <code>backup-gate.sh</code> installed, and is the <code>command=</code> path in <code>authorized_keys</code> correct?
Works by hand, fails in cron	Use absolute paths and run cron as the user who owns the key

Honest limits of this project

Backups are not encrypted on the server. Old backups on the server are not deleted automatically yet (local retention only). A graphical dashboard and failure alerts are planned for later phases.